

FreelanceHub

Plateforme de mise en relation entre Clients et Freelancers

Documentation Technique

Réalisée par : Ranim Khairallah & Nour Moussa

1. Presentation

FreelanceHub est une plateforme complete permettant aux clients de publier des projets, d'acheter des produits digitaux et de gerer des contrats avec sequestre (Escrow), tandis que les freelancers peuvent proposer leurs services, vendre des produits et travailler via un espace de travail avance (Gantt + Milestones).

Roles supportes

<i>Role</i>	<i>Description</i>
<i>client</i>	Publie des projets, gere des contrats, valide les livraisons
<i>freelancer</i>	Repond aux annonces, livre des projets, vend des produits
<i>admin</i>	Moderation globale de la plateforme

2. Fonctionnalites

Pour les Clients

- Publication d'annonces de projets
- Navigation parmi les profils freelancers
- Marketplace de produits digitaux
- Messagerie en temps reel
- Creation et suivi de contrats avec Escrow
- Validation des livraisons

Pour les Freelancers

- Gestion de profil complet
- Reponse aux annonces clients
- Vente de produits digitaux
- Espace de travail projet (Diagramme de Gantt + Milestones)
- Suivi des gains

Fonctionnalites Generales

- Authentification JWT securisee
- Chat temps reel avec indicateur de frappe
- Notifications en temps reel
- Systeme de moderation (Admin)
- Paiements via Stripe (Escrow)

3. Stack Technique

<i>Couche</i>	<i>Technologie</i>
<i>Frontend</i>	Ionic 7 + Angular (NgModule) + TypeScript + SCSS
<i>Backend</i>	Flask (Python) + Flask-SocketIO
<i>Base de donnees</i>	MongoDB Atlas (PyMongo)
<i>Authentification</i>	Flask-JWT-Extended
<i>Temps reel</i>	Flask-SocketIO + socket.io-client
<i>Emails</i>	Flask-Mail
<i>Paielements</i>	Stripe (Checkout + futur Payouts)

4. Structure du Projet

Frontend — FreelanceHub/

```
FreelanceHub/  
  src/  
    app/  
      client/  
      freelancer/  
      admin/  
      services/  
      models/  
      interceptors/
```

Backend — FreelanceHub_api/

```
FreelanceHub_api/  
  run.py           # Point d'entree unique  
  config.py        # Parametres & variables d'env  
  requirements.txt  
  seed.py          # Script de donnees de test  
  app/  
    __init__.py    # Application Factory (create_app)  
    extensions.py  # Outils globaux (db, socketio, mail)  
    models/  
      product.py  
    utils/  
      helpers.py  
    routes/  
      auth.py  
      announcements.py  
      products.py  
      contracts.py  
      messages.py  
      socket_events.py  
      admin.py
```

5. Installation & Lancement

Backend

```
cd FreelanceHub_api

# Installation des dependances
pip install flask flask-cors flask-jwt-extended flask-mail \
    flask-socketio eventlet pymongo python-dotenv

# Lancement
python run.py
```

Important : Toujours utiliser socketio.run() et non app.run() — déjà configure dans run.py.

Frontend

```
cd FreelanceHub

npm install
ionic serve
```

6. Configuration

Backend — .env

```
MONGO_URI=mongodb+srv://...
JWT_SECRET_KEY=super_secret_key
SECRET_KEY=flask_secret_key
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USERNAME=your_email@gmail.com
MAIL_PASSWORD=your_password
STRIPE_SECRET_KEY=sk ...
```

Frontend — environment.ts

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5000/api'
};
```

7. Architecture Backend

La Racine du Projet

Fichier	Role
run.py	Seul fichier a executer. Importe create_app(), cree le serveur et le demarre sur le port 5000.
config.py	Parametres de l'application (URI MongoDB, cles secretes). Utilise os + .env.
requirements.txt	Liste des packages Python (Flask, PyMongo, python-dotenv...).
seed.py	Script utilitaire a lancer une seule fois pour peupler la base avec des donnees de test.

Le Coeur du Backend — app/

Tout le code applicatif est enferme dans app/. Python reconnait ce dossier comme un module grace au fichier `__init__.py`.

A. L'Usine a Application — app/`__init__.py`

C'est le chef d'orchestre. La fonction `create_app()` execute dans l'ordre :

- Initialise Flask : `app = Flask(__name__)`
- Charge la configuration : `app.config.from_object(Config)`
- Active CORS (autorise Angular sur le port 8100/4200 a communiquer)
- Connecte la base de donnees : `init_db(app)`
- Enregistre les Blueprints (attache les URLs aux controleurs)

B. Les Outils Globaux — app/`extensions.py`

Initialise les objets globaux comme db (connexion MongoDB), socketio et mail. Ce fichier evite les importations circulaires : tout fichier de route peut faire `from app.extensions import db` proprement et sans conflit.

Les Blueprints (Controleurs)

Blueprint	Responsabilite
auth.py	Login, register, JWT
products.py	Boutique : GET /api/products
announcements.py	Annonces clients
contracts.py	Creation, paiement, livraison
messages.py	Messagerie REST
socket_events.py	Evenements temps reel
admin.py	Moderation

8. Le Voyage d'une Requete HTTP

```
[Angular]  ->  GET /api/products

      |
      v
[run.py]           Port 5000 -- recoit la requete

      |
      v
[app/___init___py] Lit l'URL, identifie le Blueprint responsable
                  "commence par /api/products -> c'est products_bp !"

      |
      v
[app/routes/products.py]  Le spécialiste traite la logique

      |
      v
[app/extensions.py]       db.products.find() -> MongoDB Atlas

      |
      v
[app/utils/helpers.py]    serialize_doc() -> convertit les ObjectId

      |
      v
[Angular]  <-  JSON propre et pret a l'affichage
```

9. Systeme de Contrats & Escrow

Cycle de Vie d'un Contrat

```
pending --> funded --> in_progress --> delivered --> completed
      |           |                               |
      Creation   Paiement                         Validation client
                Stripe                               -> Liberation fonds
                (argent bloque)
```

Repartition des Fonds a la Completion

Destinataire	Part
Freelancer	90 %
Plateforme (commission)	10 %

Fonctionnalites Avancees

- Gestion des Milestones (jalons de projet)
- Diagramme de Gantt (suivi des taches)
- Calcul automatique des commissions
- Historique complet des changements de statut

10. Socket.IO — Temps Reel

Authentification : token JWT passe dans la query de connexion.

Evenement	Description
join_conversation	Rejoindre un canal de chat
send_message	Envoyer un message
typing	Indicateur en train d'ecrire...
new_message	Reception d'un nouveau message
notification	Notifications systeme temps reel

11. Design System

Interface	Style
Client	Glassmorphism + tons indigo
Freelancer	Neumorphism orange

- Composants Ionic optimises mobile
- Experience utilisateur fluide et moderne

12. Routes API Principales

Authentification

```
POST    /api/auth/login
POST    /api/auth/register
```

Annonces

```
GET      /api/announcements
POST     /api/client/announcements
```

Contrats

```
POST     /api/contracts
POST     /api/contracts/:id/pay
PUT      /api/contracts/:id/deliver
PUT      /api/contracts/:id/validate
```

Messages

```
GET      /api/client/messages
(Socket.IO pour l'envoi en temps reel)
```

13. Known Issues & Solutions

Probleme	Solution
ID MongoDB non serialisable	Toujours utiliser <code>_id</code> (pas <code>id</code>) + <code>serialize_doc()</code>
Serveur ne démarre pas avec SocketIO	Utiliser <code>socketio.run()</code> — jamais <code>app.run()</code>
Imports circulaires	<code>socketio</code> et <code>db</code> declares dans <code>extensions.py</code>
Compatibilité ancienne API produits	Alias <code>getProducts()</code> conserve
Routes protégées non sécurisées	<code>jwt_required</code> importe dans tous les blueprints concernés

14. Améliorations Futures

- Notifications Push (FCM)
- Chat vocal / visioconférence
- Facturation PDF automatique
- Système de notation & reviews avancé
- Mode offline (PWA)
- Analytics détaillées pour l'Admin
- Migration vers Gevent (suppression des warnings Eventlet)